Wrocław University of Science and Technology
Faculty of Information and Communication Technology

Field of study: **Artificial Intelligence (SZT)**

MASTER THESIS

Analyzing esports matches using deep learning

Bartosz Tramś

Supervisor
dr hab. inż. Stanisław Saganowski, prof. uczelni

Keywords: esports, counter-strike, outcome prediction, graph neural networks, deep learning, sports analytics

WROCŁAW 2026

Abstract

This thesis investigates round win probability estimation in professional Counter-Strike 2 using deep learning. A dataset of approximately 2,300 Tier 1 professional matches from the 2025 competitive season is collected and preprocessed into a sequence of per-second game states. Seven model architectures are benchmarked across eight competitive maps, evaluated using log loss and Expected Calibration Error, with performance analysed separately for the full round and the post-plant phase. The central architectural contribution is GCNSimple, a graph convolutional network that encodes player interactions using geodesic distance and line-of-sight edge features derived from the map geometry, which achieves the best post-plant performance on seven of eight maps. The benchmark design — eight maps, phase-conditional evaluation, and tournament-boundary chronological splits — constitutes the primary methodological contribution of the work.

Streszczenie

Niniejsza praca dotyczy problemu estymacji prawdopodobieństwa wygranej rundy w profesjonalnym Counter-Strike 2 z wykorzystaniem głębokiego uczenia. Zebrano i przetworzono zbiór danych obejmujący około 2,300 meczów rozegranych na najwyższym poziomie rywalizacji w sezonie 2025, reprezentując każdą rundę jako sekwencję stanów gry próbkowanych co sekundę. Na ośmiu mapach turniejowych przeprowadzono analizę porównawczą siedmiu architektur modeli, oceniając je przy użyciu funkcji straty logarytmicznej oraz oczekiwanego błędu kalibracji, z osobną analizą wyników dla pełnej rundy oraz fazy po podłożeniu bomby. Głównym wkładem architektonicznym jest model GCNSimple — sieć splotowa na grafie kodująca interakcje między zawodnikami za pomocą cech krawędziowych opartych na odległości geodezyjnej i widoczności wzajemnej wyznaczonych z geometrii mapy, osiągająca najlepsze wyniki w fazie po podłożeniu bomby na siedmiu z ośmiu map. Głównym wkładem metodologicznym pracy jest projekt eksperymentu porównawczego — obejmującego osiem map, analizę warunkową względem fazy rundy oraz podziały chronologiczne wyznaczone granicami turniejów.

Contents

1 Introduction **1**
- 1.1 Problem Statement . . . 1
- 1.2 Thesis Objectives . . . 1
- 1.3 Thesis Outline . . . 2
- 1.4 Use of AI Tools . . . 2

2 Related Work **3**
- 2.1 Introduction to Counter-Strike . . . 3
- 2.2 Outcome prediction in sports . . . 3
  - 2.2.1 Outcome prediction in esports . . . 5
  - 2.2.2 Graph-based models in sports and esports . . . 6
- 2.3 Summary of literature . . . 6

3 Dataset **7**
- 3.1 Overview . . . 7
- 3.2 Data Collection . . . 7
- 3.3 Preprocessing . . . 7
  - 3.3.1 Round Filtering . . . 7
  - 3.3.2 Game State Sampling . . . 8
  - 3.3.3 Map-Specific Models . . . 8
- 3.4 Dataset Statistics . . . 8
- 3.5 Train/Validation/Test Split . . . 8
- 3.6 Geodesic Distance and Visibility Computation . . . 9

4 Methods **11**
- 4.1 Overview . . . 11
- 4.2 Problem Formulation . . . 11
- 4.3 State Representations . . . 11
  - 4.3.1 Vector Representation . . . 11
  - 4.3.2 Set (Player-Level) Representation . . . 12
  - 4.3.3 Graph Representation . . . 12
- 4.4 Baseline Models . . . 13
- 4.5 Vector-Based Model: LightGBM . . . 14
- 4.6 Permutation-Invariant Set Models . . . 14
  - 4.6.1 DeepSets . . . 14
  - 4.6.2 Set Transformer . . . 15

4.7 Graph Neural Network Models . . . 15
4.7.1 Graph Convolutional Network (GCNSimple) . . . 15
4.7.2 Graph Attention Network (GAT) . . . 15
4.7.3 NNConv-Based GCN (Negative Result) . . . 16
4.8 Temporal Graph Neural Network . . . 16
4.9 Training and Evaluation . . . 17
4.9.1 Loss Function . . . 17
4.9.2 Evaluation Metrics . . . 17
4.9.3 Graph Model Regularisation . . . 17
4.9.4 Normalisation . . . 18

5 Experiments **19**
5.1 Dataset and Baselines . . . 19
5.1.1 Dataset splits . . . 19
5.1.2 Baselines . . . 19
5.2 Results . . . 19
5.2.1 Overall patterns . . . 19
5.2.2 SetTransformer . . . 20
5.2.3 TemporalGNN . . . 20
5.3 Phase Analysis . . . 22
5.3.1 Post-plant advantage of graph models . . . 22
5.3.2 Late pre-plant . . . 23
5.4 Qualitative Analysis . . . 24
5.4.1 Example 1: Positional danger and the limits of tabular models . . . 24
5.4.2 Example 2: A low-probability CT win and the motivation for ECE . . . 26
5.5 Ablation Studies . . . 27
5.5.1 Ablation study: Training set size . . . 27
5.5.2 Ablation Study: Edge Features . . . 28

6 Discussion **35**
6.1 Log Loss and Calibration as Complementary Objectives . . . 35
6.2 Why LightGBM Is Harder to Beat Than Expected . . . 36
6.3 The Null Result as a Finding: Where Predictive Signal Lives . . . 36
6.4 Limitations . . . 37
6.4.1 Single-Map Training and Cross-Map Generalisation . . . 37
6.4.2 Static Graph and Absence of Temporal Reasoning . . . 37
6.4.3 Overpass as an Unreliable Data Point . . . 38

7 Conclusion **39**

1. Introduction

1.1. Problem Statement

The aim of this thesis is to predict the outcome of a round in professional Counter-Strike 2 matches using machine learning models. The model is designed to determine which team is more likely to win based on a snapshot of information available during gameplay. Such a snapshot includes the spatial positions of players, their in-game equipment, and other contextual factors that may influence the probability of a given team winning the round.

The outcome of a round is strongly influenced by player positioning, which is inherently relational. Traditional machine learning approaches may struggle to effectively capture the complex interactions between players. Therefore, this work explores the use of Graph Neural Networks (GNNs), which are well suited for modeling structured data represented as graphs. In particular, a Graph Convolutional Network (GCN) is implemented to represent players and their relationships within a round.

The proposed approach models players as nodes in a graph, while their spatial relationships are represented as edges. This representation enables the model to learn patterns related to strategic aspects of gameplay that may influence the outcome of a round.

1.2. Thesis Objectives

This thesis addresses the following objectives:

1. Constructing a large-scale dataset of professional CS2 match demonstrations across eight maps and two competitive seasons, with consistent train, validation, and test splits defined by tournament boundaries.

2. Designing three complementary game state representations — vector, set, and graph — that enable a controlled comparison across model classes sharing the same underlying player-level features.

3. Implementing and training five model classes: a gradient-boosted ensemble (LightGBM), two permutation-invariant set models (DeepSets and Set Transformer), and two graph neural network architectures (GCNSimple and GAT), with a dedicated regularisation regime for the graph models.

4. Evaluating all models using log loss and expected calibration error on a held-out test set, with phase-conditional analysis distinguishing late pre-plant and post-plant performance, and ablation studies examining data efficiency and edge feature contributions.

1.3. Thesis Outline

The remainder of this thesis is structured as follows. Chapter 2 reviews the relevant literature on sports outcome prediction, win probability modelling in esports, and graph neural networks applied to sports analytics. Chapter 3 describes the data collection and preprocessing pipeline, the structure of the CS2 demonstration dataset, and the train/validation/test split methodology. Chapter 4 details the three game state representations and the architecture, training procedure, and regularisation strategy for each model class. Chapter 5 presents the experimental results, including full-round and phase-conditional evaluation across all maps, qualitative round trace analysis, and two ablation studies. Chapter 6 interprets the results in terms of metric disagreement, the competitive performance of the vector-based baseline, and the limitations of the current approach. Chapter 7 summarises the findings and identifies directions for future work.

1.4. Use of AI Tools

During the preparation of this thesis, the **Claude** AI assistant (Anthropic, Claude Sonnet) was used as a technical and editorial aid. It was not used to generate research contributions, experimental results, or substantive conclusions.

Specific uses included:

- language editing and stylistic revision of text written by the author,
- assistance with debugging Python/PyTorch/PyG code in Google Colab training notebooks,
- help with LaTeX formatting of tables and equations,
- discussion of argumentation structure in the methods and experiments chapters.

All research decisions, model architectures, experimental designs, result analyses, and conclusions were formulated independently by the author. The AI tool was not used to generate model source code or to produce complete chapter drafts.

2. Related Work

2.1. Introduction to Counter-Strike

Counter-Strike is a series of multiplayer tactical first-person shooter video games in which two opposing teams compete to complete specific objectives within a limited time. The most recent installment, Counter-Strike 2, maintains a large professional esports ecosystem in which teams composed of salaried professional players compete in organized tournaments. These competitions may be held either on-site using a Local Area Network (LAN) environment or online when teams are geographically close enough to ensure fair latency conditions.

Matches played in professional competitions consist of several map-based games. The majority of tournaments use a best-of-three format, meaning that the first team to win two maps wins the match. Each map has a unique architectural layout, which leads to significant variation in strategy and tactical approaches between teams.

Each map is divided into a sequence of rounds played between two teams of five players each: the Counter-Terrorists (CT) and the Terrorists (T). Professional teams typically maintain fixed rosters and roles, meaning players consistently compete alongside the same teammates rather than frequently mixing with other players. At the start of every round, both teams spawn in designated areas of the map and attempt to achieve their respective objectives.

A round can be won in several ways depending on the side. The Terrorist team may win by eliminating all opposing players or by successfully planting and detonating the bomb at one of the designated bombsites. The Counter-Terrorist team may win by eliminating all Terrorist players, preventing the bomb from being planted until the round timer expires, or by defusing the bomb after it has been planted. Importantly, eliminating the opposing team is not strictly required to win a round, as objective-based outcomes frequently determine the result.

To win a map, a team must secure 13 rounds. If both teams reach twelve rounds each, the match proceeds to overtime, which is typically played in a "Max Rounds 3" (MR3) format per side until a winner is determined.

This thesis focuses specifically on the professional level of play rather than casual or amateur matches. Using data collected from professional tournaments, predictive models are developed to estimate the probability of the Counter-Terrorist side winning a round based on a snapshot of the game state at a given moment. These snapshots capture relevant contextual information such as player positions, remaining resources, and other situational variables, enabling the study of how in-round conditions influence the likelihood of victory.

2.2. Outcome prediction in sports

The application of artificial intelligence in sports analytics has grown significantly in recent years, encompassing a wide range of tasks such as action recognition, object detection

and tracking, and target classification. A comprehensive overview of these applications is provided in [8], which highlights the increasing role of AI in analyzing sports performance and supporting decision-making processes. In particular, performance analysis and outcome prediction have emerged as key research directions.

One of the fundamental approaches to predicting match outcomes relies on historical match data and temporal modeling. For instance, [15] employs a Long Short-Term Memory (LSTM) network to predict FIFA World Cup match results using historical match logs and team ranking coefficients. The study demonstrates that incorporating temporal dependencies leads to improved predictive performance compared to traditional methods such as logistic regression, highlighting the importance of sequential modeling in sports analytics.

In [22], player statistics from the 2010 and 2014 FIFA World Cups are used as input to deep neural networks. The authors construct separate models for different tournament stages and player positions, optimizing architecture and hyperparameters through grid search. Their results indicate strong predictive performance both across the group stage and play-off matches. Notably, the study emphasizes the relative importance of individual performance indicators, addressing a gap in traditional approaches that often overlook player-level contributions. While the study highlights the predictive value of individual performance indicators, this thesis takes a complementary approach: rather than modelling player identity or historical statistics, the focus is on the relational structure of the game state at a given moment, capturing how players are positioned and interactrelative to one another within a single round

More recent work explores the use of graph-based representations to capture interactions between players. In [5], a graph-based framework is proposed for event prediction in soccer, where nodes represent multimodal features, and edges encode relationships between players based on their positions and team affiliations. The model combines a two-layer Graph Convolutional Network (GCN) with a GRU-based recurrent architecture and probabilistic modeling, achieving a high average precision of 0.948. This approach demonstrates the effectiveness of incorporating spatial-temporal interactions and relational structures into predictive models.

In addition to structured match data, computer vision techniques are increasingly used to extract information directly from video recordings. For example, [25] introduces a multi-component system combining vision-based feature extraction with large language model (LLM)-based agents to analyze badminton matches. The system leverages architectures such as UNet, YOLOv5, and TSM-ResNet50 to identify player locations, actions, and gestures with high accuracy, and subsequently transforms this information into structured insights. Similarly, [2] utilizes convolutional and recurrent neural networks to recognize key in-game actions in basketball, enabling real-time feedback for coaching purposes.

However, it is important to note a fundamental distinction between traditional sports and esports. While many sports analytics approaches rely heavily on video-based data extraction, esports environments provide direct access to highly accurate, fine-grained event data generated by the game engine itself. This eliminates the need for complex visual processing pipelines and enables more precise modeling of player behavior and interactions. Consequently, esports analytics can leverage richer and more reliable datasets, opening new opportunities for advanced predictive modeling.

2.2.1. Outcome prediction in esports

Given the relatively limited number of comprehensive analytical frameworks in esports, several recent studies stand out as particularly relevant to the problem of outcome prediction. This section highlights three key works that form the foundation for the approach adopted in this thesis.

One of the most influential contributions is presented in [18], where the authors address the lack of standardized player evaluation methods in Counter-Strike. The study introduces a context-aware framework for valuing player actions based on their impact on the team's probability of winning a given round. Central to this framework is a win probability model trained on game-state snapshots using gradient-boosted tree ensembles; the authors consider logistic regression, CatBoost, and XGBoost, noting that they "focus on a selection of models that can produce well calibrated probabilities," with XGBoost achieving the best performance across all reported metrics. From this model, the authors derive the win probability added (WPA) metric, which captures the contribution of individual player actions by measuring how each action shifts the team's estimated win probability. The work is widely regarded as a foundational contribution to esports analytics and has influenced later metrics used in professional analysis platforms. Notably, the win probability model operates on a unified feature vector aggregating team-level statistics across all maps, using the map identity as a single categorical feature. The present thesis extends this line of work by training separate map-specific models and replacing the tabular representation with permutation-invariant set models and graph neural networks that explicitly encode spatial relationships between players.

Another important contribution is [20] which focuses on data availability and problem formulation in esports. The authors introduce the Esports Trajectory and Actions (ESTA) dataset, one of the largest publicly available datasets in sports analytics, consisting of millions of player actions, trajectories, and game states extracted directly from game logs. In addition to providing a valuable resource, the study formalizes the round outcome prediction problem as estimating the probability $P(Y_i = 1|g_i)$ where the game state g_i represents the full context of a round at a given state. This formulation is relevant, as it is the predictive task considered in this thesis.

The third key work, [21], addresses a fundamental limitation of traditional modeling approaches, namely the inability to effectively capture interactions between players. The authors propose a sport-agnostic graph-based representation of game states, where players are modeled as nodes and their relationships as edges. This representation is used as input to Graph Neural Networks, enabling the model to preserve permutation invariance and learn complex interaction patterns. The approach demonstrates significant improvements over state-of-the-art methods, reducing prediction error in both American football and esports scenarios. Additionally, the framework allows for interpretability through the analysis of player relationships and supports counterfactual reasoning.

Together, these three works highlight the evolution of esports analytics from simple aggregate statistics, through context-aware evaluation metrics, to advanced representations that explicitly model player interactions. They also reveal a clear trend toward leveraging high-resolution, event-level data and more expressive machine learning models.

2.2.2. Graph-based models in sports and esports

Graph-based models have emerged as a powerful approach for analyzing complex relational data in sports and esports. Unlike vector-based methods that represent each player or team independently using aggregated features, graph representations model interactions between entities. In this setting, players are represented as nodes, while edges encode relationships between them such as spatial proximity. The main advantage of this approach lies in its ability to preserve relational structure. In many sports scenarios, the outcome of the play is not solely determined by individual performance, but rather by interactions between players. Graph Neural Networks enable the learning of representations that incorporate information from neighboring nodes through message passing mechanisms - they can capture both local and global dependencies within a team or between opposing teams.

This type of modeling has been used in a variety of sports. In soccer, GCNs have been utilized to predict game outcomes based on player passing networks [13], while [17] used GNNs to model behavior of defenders as well as its effect on the attackers. Modeling soccer as a series of events related to the ball [4] and the development of continuous modeling techniques for american football [23] highlight the complexity of input features that is able to be efficiently analyzed. The application of these models extends to esports, such as *League of Legends*, where GCNs were used to predict entire season outcomes [1]. More advanced models like Spatio-Temporal Graph Neural Networks have been developed for specific analysis, seen in generalized spatial-temporal graph analysis review [6] and applied to actions happening on a soccer pitch [14].

2.3. Summary of literature

The reviewed literature demonstrates significant progress in the application of machine learning to sports and esports analytics. Early approaches focused on aggregated, vector-based representations of player performance, while more recent work incorporates richer spatiotemporal data and advanced models such as deep neural networks and graph-based methods.

While previous studies have demonstrated the effectiveness of graph neural networks in modeling sports data, their application in esports remains relatively limited and has not fully exploited the richness of available event-level information. Existing graph-based approaches typically define relationships using proximity-based interaction schemes, which may not accurately reflect the true dynamics of gameplay.

To address these gaps, this thesis proposes a novel graph-based representation of game states in Counter-Strike, where edges between players are defined based on visibility relationships rather than simple spatial proximity. This allows for a more accurate modeling of tactical interactions, as visibility plays a critical role in decision-making and engagement outcomes in FPS games. By combining this representation with graph neural networks and event-level data, the proposed approach aims to improve the modeling of round outcomes and provide a more realistic representation of in-game dynamics.

3. Dataset

3.1. Overview

This chapter describes the dataset used to train and evaluate the predictive models proposed in this thesis. The dataset consists of professional Counter-Strike 2 match replay files collected from the highest tier of competitive play and parsed into a structured format suitable for machine learning. The construction of this dataset closely follows the methodology introduced in [20], extending it to the CS2 environment and expanding its scope to cover the full 2025 competitive year.

3.2. Data Collection

Demo files were collected from HLTV.org, a widely used platform for professional Counter-Strike match data. Only matches from tournaments classified as S-tier on Liquipedia[1] or as Tier 1 by VALVe, the game's publisher, were included. This restriction ensures that the dataset captures high-level strategic play and minimizes noise that may arise from lower-quality competition. The data spans the full 2025 calendar year, covering two complete competitive CS2 seasons.

In total, the dataset comprises **2388** maps played across all maps present in the active competitive pool during the collection period. Demo files were parsed using `awpy` [20], an open-source Python library for Counter-Strike demo parsing. The `awpy` library extracts structured information from the binary demo format, exposing player-level spatial and event data at each server tick.

3.3. Preprocessing

3.3.1. Round Filtering

Not all rounds present in a raw demo file are suitable for model training. Knife rounds, which are used at the start of certain maps to determine side selection, do not reflect normal gameplay dynamics and were excluded from the dataset. Additionally, rounds that terminated due to a server restart or other technical irregularity, rather than a legitimate in-game win condition, were removed. This filtering step ensures that each sample in the dataset corresponds to a genuine competitive round with a well-defined outcome.

[1] https://liquipedia.net/counterstrike

3.3.2. Game State Sampling

Each retained round is decomposed into a sequence of game states. A game state is recorded at two types of moments: once per second of elapsed round time, and at every server tick at which a damage event occurred. The first sampling strategy captures the gradual evolution of positional and economic conditions throughout the round, while the second ensures that high-impact events — such as player eliminations — are represented at full temporal resolution. This dual-rate sampling scheme reflects the observation that most meaningful state changes in a CS2 round are either slow-moving (economy, positioning) or event-driven (damage, deaths).

3.3.3. Map-Specific Models

Each competitive map in Counter-Strike 2 has a distinct architectural layout, which gives rise to fundamentally different tactical structures, common engagement areas, and strategic conventions. Accordingly, all models in this thesis are trained and evaluated separately for each map, using only data collected from matches played on that map. This design choice follows [19], where map-specific models consistently outperform unified models trained across all maps. Separate models also simplify ablation analysis, as performance differences can be attributed to map-specific phenomena rather than confounded by cross-map variance.

The active map pool changed between the two 2025 competitive seasons. Six maps — `de_dust2`, `de_mirage`, `de_inferno`, `de_nuke`, `de_ancient`, and `de_train` — were present throughout the entire year. `de_anubis` was part of the pool only during Season 1, while `de_overpass` replaced it for Season 2, which began after the player break ending on July 14, 2025. Because per-map models are trained exclusively on data from their respective map, this asymmetry is handled naturally: the `de_anubis` models are trained solely on Season 1 data, and the `de_overpass` models solely on Season 2 data, with no special treatment required.

3.4. Dataset Statistics

The final dataset contains over two thousand maps spanning all eight maps present in the competitive pool during the data collection period. Table 3.1 summarizes the per-map breakdown of the dataset. Maps marked with a season annotation were active only during the indicated portion of the year.

3.5. Train/Validation/Test Split

To simulate a realistic deployment scenario — in which a model is trained on historical data to predict outcomes in future, unseen tournaments — a chronological split was applied rather than a random partition. Random splits would introduce temporal leakage, as game states from the same round or tournament would appear in both the training and test sets, inflating estimated performance. The split for 6 maps which were in the map pool for the entirity of 2025 is organised as follows:

Table 3.1: Per-map dataset statistics. Maps active for only one season are annotated accordingly.

Map	Season	Maps	Rounds	Game States
de_dust2	S1 + S2	404	8713	993893
de_mirage	S1 + S2	381	8449	933568
de_inferno	S1 + S2	429	9339	1134718
de_nuke	S1 + S2	368	8035	902127
de_ancient	S1 + S2	345	7503	814793
de_train	S1 + S2	206	4559	504824
de_anubis	S1 only	156	3320	387968
de_overpass	S2 only	99	2145	255788
Total		–	–	–

- **Training set:** Maps played from January through September 2025, spanning the entirety of Season 1 and the majority of Season 2.
- **Validation set:** Maps played in October and November of 2025, used for hyperparameter tuning and early stopping.
- **Test set:** Maps covering the StarLadder Budapest Major, which concluded on December 14, 2025.

This chronological structure ensures that the test set contains matches from a tournament period entirely unseen during training, providing a conservative and realistic estimate of model generalisation. The test set is anchored to a single high-profile tournament — a Major, the most prestigious event format in competitive Counter-Strike — which further ensures that the evaluation reflects top-tier play rather than a mixture of event tiers. For the two season-exclusive maps, `de_anubis` and `de_overpass`, the split is applied within the data available for each map independently, preserving the chronological ordering while respecting the boundaries of the seasons in which they were active.

3.6. Geodesic Distance and Visibility Computation

Two key edge features used in the graph-based models, geodesic distance and line-of-sight visibility, are derived from the map navigation mesh provided by the `nav` module of `awpy`. The navigation mesh is a graph structure used by the game engine's bot AI to reason about reachability on the map. It encodes the walkable surface of the map as a set of convex polygons, enabling the computation of shortest walkable paths between arbitrary points.

Geodesic distance between two players is computed as the shortest path distance on this navigation mesh, rather than the Euclidean distance between their coordinates. This is a critical distinction in the CS2 environment, where maps contain multiple vertical levels, narrow corridors, and obstacles that make straight-line distance a poor proxy for true spatial

proximity. Two players separated by a thin wall may be far apart in tactical terms, yet adjacent in Euclidean space.

Line-of-sight visibility between players is computed as a binary indicator: a value of 1 is assigned if player i has an unobstructed view of player j at the given game state, and 0 otherwise. This information is derived from the navigation mesh structure and reflects the tactical notion of whether two players could directly engage one another. As noted in [**?**], visibility-based edges provide a more tactically meaningful relational structure than proximity-based ones, since engagement in CS2 is fundamentally constrained by line of sight rather than physical distance.

4. Methods

4.1. Overview

This chapter describes the predictive models developed and evaluated in this thesis. The goal is to estimate the probability that the Counter-Terrorist side wins a given round, conditioned on a snapshot of the game state observed at a particular moment during that round. Section 4.2 formalises the prediction task. Sections 4.3 through 4.8 describe the three classes of model evaluated: a vector-based gradient-boosted tree, permutation-invariant set models, and graph neural networks. Section 4.9 covers shared training and evaluation details.

4.2. Problem Formulation

Each round r is associated with a binary outcome $Y_r \in \{0, 1\}$, where $Y_r = 1$ denotes a CT-side victory and $Y_r = 0$ denotes a T-side victory. During a round, the game progresses through a sequence of game states $s_1, s_2, \ldots, s_T$, where each state s_t captures the full configuration of all alive players at server tick t: their positions, health, equipment, and other contextual attributes. The prediction task is to approximate the function

$$f(s_t) = P(Y_r = 1 \mid s_t), \tag{4.1}$$

which assigns a win probability to the CT side given the observed game state. This formulation is consistent with the problem statement introduced in [20] to evaluate CS:GO games and is extended to CS2 in this work.

Because every game state within the same round shares the same label Y_r, models are evaluated on a per-state basis but trained to predict the round outcome from any given state. This design choice enables continuous, per-second win probability estimates throughout the course of a round, which is useful both for analysis and for the qualitative evaluation discussed in Chapter 5.

4.3. State Representations

To enable a fair comparison across different modelling paradigms, each game state s_t is encoded using three complementary representations, each tailored to a specific model class.

4.3.1. Vector Representation

In the vector-based setting, each game state is encoded as a fixed-length feature vector $\mathbf{v}_t \in \mathbb{R}^d$ constructed by aggregating player-level information into team-level statistics. Specifically, the following features are computed separately for the CT and T sides and concatenated: the

number of alive players, the total remaining health and armour, the total current equipment value, the number of players holding a defuser, a binary flag indicating whether a player on that side is carrying the bomb, and counts of players holding each of nine weapon categories. Global features appended to this representation include a binary bomb-planted flag, the active bombsite indicator, and the number of seconds elapsed since the current phase began. Player identities and positions are discarded entirely in this representation. The resulting vector is compact and has a fixed dimensionality regardless of how many players remain alive, making it well-suited for tree-based methods.

4.3.2. Set (Player-Level) Representation

In the set-based setting, a game state is represented as a variable-size set of player feature vectors $\mathbf{X}_t = \{\mathbf{x}_t^p\}_{p \in \mathcal{A}_t}$, where $\mathcal{A}_t$ denotes the set of alive players at tick t and each $\mathbf{x}_t^p \in \mathbb{R}^{32}$ encodes the individual state of player p. The cardinality of $\mathbf{X}_t$ varies between 2 and 10 as players are eliminated during the round.

Table 4.1 lists the 32 components of $\mathbf{x}_t^p$. Positional coordinates are normalised to the axis-aligned bounding box of the map, velocity components are divided by 300 (the approximate maximum speed in CS2), and health and armour values are divided by 100. Equipment value is divided by 16,000, corresponding to the maximum possible loadout cost. The elapsed time since the current phase start is divided by 115 (the maximum pre-plant round duration in seconds), and if the bomb is planted during the round, the timer is divided by 40 (the bomb timer duration). Flash duration is included in its raw form (seconds) and is not normalised, as it has no fixed upper bound in the game engine and values are typically close to zero outside of active flash events.

The nine weapon categories used in this work represent a finer-grained taxonomy than prior approaches. The ESTA benchmark [20] encodes weapon information using three broad binary flags — rifle, SMG, and heavy — which collapse the full breadth of CS2's arsenal into coarse classes. The categorisation proposed here distinguishes between: base rifles split by side (AK-47 for T; M4A4/M4A1-S for CT), economy rifles (Galil AR, FAMAS), premium snipers (AWP, SCAR-20, G3SG1), economy snipers (SSG 08), SMGs (including the Negev, which is sufficiently rare to not warrant a separate category), shotguns, and two tiers of pistols (premium and base). This granularity allows the model to differentiate between, for example, a player holding an AWP and one holding an FAMAS — a tactically significant distinction that a single binary rifle flag would obscure. The same nine categories are applied consistently in both the vector and set representations, ensuring that weapon information is encoded at the same resolution across all model classes.

4.3.3. Graph Representation

In the graph-based setting, the game state is encoded as a directed fully-connected graph $G_t = (\mathcal{V}_t, \mathcal{E}_t)$. Each alive player $p \in \mathcal{A}_t$ is represented as a node $v_p \in \mathcal{V}_t$ with the same 32-dimensional feature vector $\mathbf{x}_t^p$ used in the set representation. An edge (i, j) is included for every ordered pair of distinct alive players, making the graph fully connected. Each edge carries a 3-dimensional feature vector $\mathbf{e}_{ij} \in \mathbb{R}^3$:

Table 4.1: Player-level feature vector $\mathbf{x}_t^p \in \mathbb{R}^{32}$. All continuous features are normalised prior to training.

Index	Feature	Normalisation
0–2	Position (x, y, z)	Map bounding box
3–5	Velocity (v_x, v_y, v_z)	$\div 300$
6–7	View angles (pitch, yaw)	$\div 90$, $\div 180$
8	Health	$\div 100$
9	Armour	$\div 100$
10	Side ($\mathbb{1}$[CT])	Binary
11	Has helmet	Binary
12	Has defuser	Binary
13	Is bomb carrier	Binary
14	Flash duration	Raw (seconds)
15	Equipment value	$\div 16{,}000$
16–24	Weapon category (9 bins)	Binary flags
25	Seconds since phase start	$\div 115$
26	Seconds since bomb plant	$\div 40$
27	Bomb planted	Binary
28	Bomb at site A	Binary
29	Bomb at site B	Binary
30	Active smoke count	$\div 10$
31	Active molotov/inferno count	$\div 10$

$$\mathbf{e}_{ij} = \Big[\, w_{ij}^{\text{geo}},\ \mathbb{1}[\text{vis}_{ij}],\ \mathbb{1}[i \text{ and } j \text{ on same team}] \,\Big], \tag{4.2}$$

where $w_{ij}^{\text{geo}} = \min\Big(1/(d_{ij}^{\text{geo}} + \varepsilon),\ 10\Big)$ is the inverse geodesic distance between players i and j on the map navigation mesh, clamped to a maximum of 10. The parameter $\varepsilon = 10^{-6}$ prevents division by zero. $\mathbb{1}[\text{vis}_{ij}]$ is a binary line-of-sight indicator, and the same-team flag indicates whether the two players are on the same side. The rationale for using inverse geodesic distance rather than raw distance is that nearby players should exert stronger influence on one another during message passing. Geodesic distance is preferred over Euclidean distance because the maps in CS2 contain multiple vertical levels, walls, and narrow corridors that make straight-line distance a poor proxy for actual tactical proximity, as discussed in Section 4.5 of [20].

4.4. Baseline Models

Two rule-based baselines are included as lower-bound performance references.

The **constant predictor** always predicts a CT win with probability 1.0. Its log loss is $-\log(p_{\text{CT}})$, where p_{CT} is the empirical CT win rate on the test set, and its expected calibration error (ECE) equals $|p_{\text{CT}} - 1.0|$.

The **health-difference predictor** computes the scalar difference $\Delta h_t = H_t^{\mathrm{CT}} - H_t^{\mathrm{T}}$ between the total remaining health of both sides and converts it to a probability via the logistic function: $\hat{p}_t = \sigma(\Delta h_t / \tau)$, where τ is a temperature parameter fitted on the validation set. This baseline captures a well-known strong signal — as identified in [18] — without access to positional or economic features.

4.5. Vector-Based Model: LightGBM

The primary vector-based model is a gradient-boosted decision tree ensemble implemented with LightGBM [9]. The model takes the vector representation $\mathbf{v}_t$ as input and is optimised using binary cross-entropy (log loss) as the objective. Feature subsampling and bagging are applied at each boosting round to reduce overfitting: a fraction of 0.9 of features and 0.8 of training samples are selected at random; bagging is performed every 5 boosting rounds. Training proceeds for up to 500 boosting rounds, with early stopping triggered if the validation log loss does not improve for 10 consecutive rounds.

LightGBM represents the state-of-the-art approach for tabular data in this domain and has been shown to be a strong baseline in prior work [18, 20]. It serves as the primary point of comparison for the neural models.

4.6. Permutation-Invariant Set Models

Both set-based models take the variable-size player set $\mathbf{X}_t$ as input and are required to be permutation-invariant: the output probability must not depend on the order in which players are presented. This is a necessary property since there is no canonical ordering of players in a CS2 game state. Both models are trained with the Adam optimiser [10] at a learning rate of 10^{-3} and weight decay of 10^{-4}, with gradient clipping at a norm of 1.0 and early stopping with patience 10.

4.6.1. DeepSets

DeepSets [24] achieves permutation invariance by applying a shared encoder ϕ to each player independently, averaging the resulting representations, and passing the aggregate through a decoder ρ:

$$\hat{p}_t = \rho\left(\frac{1}{|\mathcal{A}_t|}\sum_{p\in\mathcal{A}_t}\phi(\mathbf{x}_t^p)\right). \tag{4.3}$$

The encoder ϕ consists of two linear layers with a ReLU activation and Dropout (rate 0.3) after the first layer, mapping from $\mathbb{R}^{32}$ to $\mathbb{R}^{128}$. The decoder ρ has the same structure, mapping from $\mathbb{R}^{128}$ to a scalar logit via a two-layer MLP with ReLU and Dropout (rate 0.3) after the first layer. The logit is converted to a probability via the sigmoid function at inference time. Batches are grouped by the number of alive players to avoid padding, following the approach used in [20].

4.6.2. Set Transformer

The Set Transformer [12] extends the DeepSets framework by replacing simple mean aggregation with an attention-based mechanism, enabling the model to capture interactions between players prior to pooling. The architecture consists of three stages. An initial linear layer followed by a ReLU activation projects each player vector from $\mathbb{R}^{32}$ to $\mathbb{R}^{128}$. The encoder applies one Induced Set Attention Block (ISAB) with 4 inducing points and a single attention head; dropout (rate 0.1) is applied within each attention and feedforward sublayer. The decoder applies one Pooling by Multihead Attention (PMA) block with 1 seed vector and 1 attention head, producing a single summary vector. A final linear layer preceded by Dropout (rate 0.1) maps this summary to the output logit. The ISAB and PMA blocks are implemented via concatenation-based self-attention over the union of inducing points and input tokens, following the computational pattern of the original formulation [12].

4.7. Graph Neural Network Models

Graph-based models take the graph G_t as input and apply message-passing operations over the player graph prior to prediction. All graph models are trained with the same optimiser configuration as the set models.

4.7.1. Graph Convolutional Network (GCNSimple)

The primary graph model is a two-layer graph convolutional network built using the GCNConv operator [11], which computes node representations as

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \frac{w_{ij}}{\sqrt{d_i d_j}}\,\mathbf{h}_j^{(l)}\mathbf{W}^{(l)}\right), \tag{4.4}$$

where $w_{ij} = w_{ij}^{\text{geo}}$ is the inverse geodesic distance scalar edge weight, d_i and d_j are the degree normalization terms, and $\mathbf{W}^{(l)}$ is a learnable weight matrix for layer l. Because GCNConv accepts only a scalar edge weight rather than a vector, the visibility and same-team edge features are incorporated into the node representation instead. For each player i, the mean visibility and mean same-team indicators are computed over all outgoing edges and concatenated to the player feature vector, expanding the input dimensionality from 32 to 34.

Each GCNConv layer is followed by Batch Normalisation and a ReLU activation, with a hidden dimensionality of 32. DropEdge [16] with probability 0.5 is applied during training to regularise the graph structure. After two message-passing layers, node representations are aggregated via global mean pooling to produce a single graph-level embedding, which is passed through a two-layer MLP (32→32→1) with Dropout (rate 0.5) between the linear layers and a sigmoid output.

4.7.2. Graph Attention Network (GAT)

Multi-head attention with 4 heads and a per-head dimensionality of 8 is used; the outputs are concatenated to produce a 32-dimensional representation. Two such GATConv layers

are stacked, each followed by Batch Normalisation and a ReLU activation. DropEdge with probability 0.5 and attention dropout with rate 0.3 are applied during training, followed by the same MLP head structure as GCNSimple (32→32→1), Dropout rate 0.3 between layers). A five-epoch linear warmup at learning rate 10^{-4} precedes the main training phase, after which a `ReduceLROnPlateau` scheduler halves the learning rate on validation loss plateau. Unlike GCNSimple, which incorporates visibility and same-team information through node augmentation prior to convolution, GAT receives the full 3-dimensional edge feature vector $\mathbf{e}_{ij}$ directly as input to the attention mechanism via the `edge_dim` parameter of GATConv, allowing the attention weights themselves to be conditioned on relational edge information.

4.7.3. NNConv-Based GCN (Negative Result)

An initial graph model was implemented using the NNConv operator [**?**], which generalises the convolution in Equation (2) by replacing the fixed weight matrix $\mathbf{W}$ with an edge-conditioned matrix produced by a small neural network:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \Theta(\mathbf{e}_{ij})\,\mathbf{h}_j^{(l)}\right), \tag{4.5}$$

where $\Theta : \mathbb{R}^3 \to \mathbb{R}^{d_{\text{in}}\times d_{\text{out}}}$ is a learned MLP that maps the 3-dimensional edge feature vector to a full weight matrix. With $d_{\text{in}} = 32$ and $d_{\text{out}} = 128$, this requires Θ to produce a $32 \times 128 = 4{,}096$-dimensional output from only 3 input values. In practice, this mapping is severely underdetermined: the small edge MLP cannot produce meaningfully different weight matrices for structurally distinct edges, and the resulting gradients do not propagate usefully through the convolution. The model was observed to converge to a constant prediction of 0.5 (log loss $\approx \ln 2 \approx 0.693$) across all training epochs, with no improvement under Xavier initialisation with reduced gain. This failure mode is consistent with the architectural constraints of the NNConv operator when edge features are low-dimensional relative to the required output matrix size [**?**]. The model is reported as a negative result and is not used for further analysis. The GCNSimple architecture described in Section 4.7.1 was developed as a direct replacement.

4.8. Temporal Graph Neural Network

To investigate whether explicitly modelling the temporal evolution of a round improves prediction, a Temporal GNN is evaluated. The architecture consists of two components. A graph encoder sharing the same GCNConv-based structure as GCNSimple (Section 4.7.1) produces a fixed-dimensional graph embedding $\mathbf{z}_t \in \mathbb{R}^{32}$ for each game state s_t in the sequence, applying Dropout after each convolutional layer prior to pooling. A single-layer Gated Recurrent Unit (GRU) [3] then processes the sequence of embeddings $(\mathbf{z}_1, \ldots, \mathbf{z}_T)$ in order, producing a hidden state $\mathbf{u}_t$ at each step. A linear head maps each $\mathbf{u}_t$ to the output logit, enabling per-step win probability estimates from a model that has access to the full history of the round up to tick t.

Each round constitutes one training sequence, padded and packed to a maximum length of 80 game states due to memory constraints. Labels are the round outcome Y_r, replicated at

every step. A learning rate of 3×10^{-4} and gradient clipping at norm 0.5 were found necessary to partially stabilise training, as the combination of graph-level message passing and GRU backpropagation through variable-length sequences produces high-variance gradients.

A key limitation of this implementation is that the graph encoder weights are frozen during training: embeddings are produced inside a `torch.no_grad()` context with DropEdge disabled, so no gradient signal flows back through the GCNConv layers. The GRU is therefore trained on fixed, detached embeddings rather than representations that are jointly optimised for temporal prediction. The consequences of this design choice are discussed in Section 5.2.3.

4.9. Training and Evaluation

4.9.1. Loss Function

All models are trained to minimise binary cross-entropy (log loss):

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^{N} \left[Y_r \log \hat{p}_t + (1 - Y_r) \log(1 - \hat{p}_t) \right], \tag{4.6}$$

where N is the number of game states in the training set, Y_r is the binary round outcome, and $\hat{p}_t$ is the predicted CT win probability at state t. All game states within a round share the same label Y_r.

4.9.2. Evaluation Metrics

Two metrics are reported. Log loss (binary cross-entropy) is the primary metric, as it directly evaluates the quality of the predicted probabilities rather than hard classification decisions. Lower log loss indicates better-calibrated and more confident correct predictions.

Expected Calibration Error (ECE) [7] measures the degree to which the predicted probabilities agree with observed win frequencies. The probability range $[0, 1]$ is partitioned into $M = 10$ equal-width bins. For each bin m, the absolute difference between the mean predicted probability $\bar{p}_m$ and the observed win rate $\bar{y}_m$ is computed and weighted by the fraction of samples falling in that bin:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \bar{y}_m - \bar{p}_m \right|, \tag{4.7}$$

where $|B_m|$ is the number of samples in bin m. A low ECE indicates that the model's stated confidence is well-calibrated with respect to actual outcomes.

4.9.3. Graph Model Regularisation

Graph neural network models overfit substantially faster than the set and vector-based models on single-map datasets of the scale used in this work. A dedicated regularisation configuration (`GRAPH_CFG`) is therefore applied exclusively to GCNSimple and GAT, leaving the hyperparameters of all other models unchanged. Table 4.2 summarises the configuration.

Table 4.2: Graph model regularisation configuration (`GRAPH_CFG`) applied to GCNSimple and GAT across all maps.

Parameter	Value
`hidden`	32
`dropout_gcn`	0.5
`dropout_gat`	0.3
`drop_edge_p`	0.5
`weight_decay`	10^{-3}
`patience`	5
`lr_warmup_gat`	10^{-4} (5 epochs, GAT only)

Parameters were determined empirically on de_dust2 and applied without further per-map tuning; the `dropout_gat` rate is lower than `dropout_gcn` because aggressive attention dropout causes training loss to exceed validation loss on this architecture, indicating underfitting of the attention weights. In addition, a `ReduceLROnPlateau` scheduler with reduction factor 0.5 and patience 3 is applied to both models, halving the learning rate whenever validation loss fails to improve for three consecutive epochs.

4.9.4. Normalisation

A per-feature normaliser is fitted on the training set alone and applied to the set and graph representations prior to model training. For each of the 32 player-level features, the training set mean and standard deviation are computed over all players in all training game states. These statistics are saved and applied identically to the validation and test sets, ensuring no information leakage. The vector representation used by LightGBM is not normalised, as gradient-boosted trees are invariant to monotone feature transformations. ll

5. Experiments

5.1. Dataset and Baselines

5.1.1. Dataset splits

Table 5.1 summarises the dataset splits across all eight maps. For maps with data from both collection seasons, the held-out test set is anchored to the StarLadder Major — a single high-profile tournament that provides a consistent, top-tier evaluation benchmark across maps. For `de_anubis` and `de_overpass`, which appear in only one collection season, a chronological within-season split is used instead. The `de_overpass` training set contains only 67 matches, which is notably smaller than all other maps; results on this map should be interpreted with caution.

Table 5.2 reports the CT win rate and corresponding Constant baseline log loss on each map's test set. Because the model is trained and evaluated at the game-state level rather than the round level, the CT win rate here is the proportion of *game states* (not rounds) in which the CT side ultimately won the round. Maps vary from favouring the T-side (`de_anubis`, CT win rate 44.1%) to mildly CT-favoured (`de_nuke`, 53.4%). This variation in base rates is relevant when comparing Constant baseline log losses across maps: a lower Constant LL indicates a more balanced map, not a harder prediction problem.

5.1.2. Baselines

The two rule-based baselines used throughout this chapter — the Constant predictor and the HP-diff predictor — are defined in Section 4.4.

5.2. Results

Tables 5.3 and 5.4 report full-round test set log loss (LL) and expected calibration error (ECE) for all models across all eight maps.

5.2.1. Overall patterns

Across the seven maps with sufficient training data, GCNSimple achieves the lowest full-round log loss among the models on four maps (`de_dust2`, `de_nuke`, `de_anubis`, and `de_inferno`). DeepSets led on `de_mirage` and `de_train`, while LightGBM achieved the lowest log loss on `de_ancient`, though GCNSimple trailed by only 0.0015 — the smallest first-to-second margin across the map pool outside of `de_inferno`. SetTransformer consistently underperforms relative to the other learned models, particularly on `de_nuke`, `de_ancient`, and `de_anubis`, where its log loss exceeds 0.49.

Calibration performance is more mixed across maps. GAT achieved the lowest ECE on `de_ancient` (0.021) and `de_anubis` (0.016); on `de_nuke`, GCNSimple and GAT were equal (both 0.021). No single architecture dominates calibration across all maps, although all learned models substantially outperform the HP-diff baseline.

A Friedman test on the full-round log loss rankings confirms that the observed differences between models are unlikely to be attributable to chance ($\chi^2 = 20.92$, $p = 0.0003$). Post-hoc Nemenyi tests show that GCNSimple significantly outperforms SetTransformer ($p = 0.0013$) and GAT ($p = 0.02$), while DeepSets outperforms SetTransformer. However, the differences between GCNSimple, DeepSets, and LightGBM are not statistically significant, indicating that these three models form a statistically indistinguishable top-performing group on the full-round prediction task. GAT, despite ranking significantly below GCNSimple on log loss, achieves the best or joint-best ECE on three of the seven maps.

The `de_overpass` results are excluded from comparative discussion throughout this chapter. With only 67 training matches, validation loss was unstable during model selection, and the test-set metrics show behaviour not observed on any other map. In particular, SetTransformer records an ECE of 0.114, substantially worse than its calibration on maps with larger training sets. These results are reported for completeness but are not considered reliable evidence when comparing model performance.

5.2.2. SetTransformer

SetTransformer fails to train meaningfully on any map. Across all eight maps, the best validation checkpoint is reached at epoch 1 in seven cases and epoch 2 in one (`de_nuke`), after which validation loss diverges from training loss while training loss continues to fall. Figure 5.1 plots the gap between validation and training log loss at each epoch; a rising curve indicates the model is memorising training trajectories rather than generalising. On `de_anubis`, the val–train gap reaches 0.72 by epoch 11 — the largest in the study — consistent with its small training set of 107 matches. On `de_overpass` (67 matches), the val–train gap reaches 1.24 by epoch 11 and the final validation LL exceeds the HP-diff baseline.

This behaviour is consistent with the known data requirements of the Induced Set Attention Block (ISAB) architecture [12]: the cross-attention mechanism introduces a large number of parameters relative to the number of training rounds available per map, and the model is unable to learn a stable set encoding before overfitting. The result is not specific to any map or split; it is a mismatch between architecture capacity and dataset size. The best-epoch checkpoints still produce competitive log losses (within 0.01–0.04 of LightGBM on most maps), suggesting the initial representations are informative, but calibration degrades noticeably: on `de_nuke`, SetTransformer ECE reaches 0.041, which more than 60% higher than LightGBM's 0.025.

5.2.3. TemporalGNN

The TemporalGNN — a GRU applied over an 80-tick sliding window of GCNSimple graph embeddings — was evaluated on `de_dust2` only. Its test log loss of 0.527 is substantially worse than the static GCNSimple (0.454) and even the HP-diff baseline on most other maps (Figure 5.2)

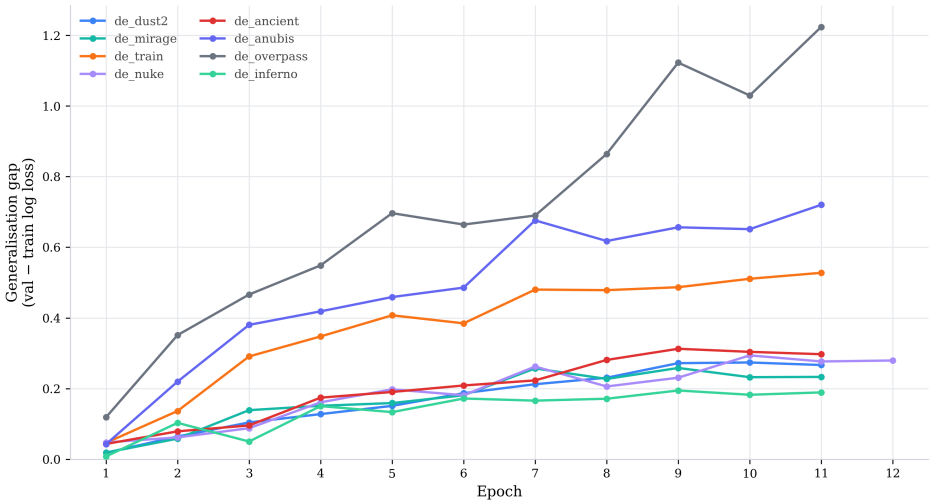


Figure 5.1: Gap between validation and training log loss (val LL − train LL) at each SetTransformer training epoch across all eight maps. A rising gap indicates overfitting: the model memorises training trajectories while failing to generalise. `de_overpass` and `de_anubis` show the most extreme divergence, consistent with their small training sets (67 and 107 matches respectively). `de_nuke` is the only map where the gap remains near zero at epoch 2 before diverging

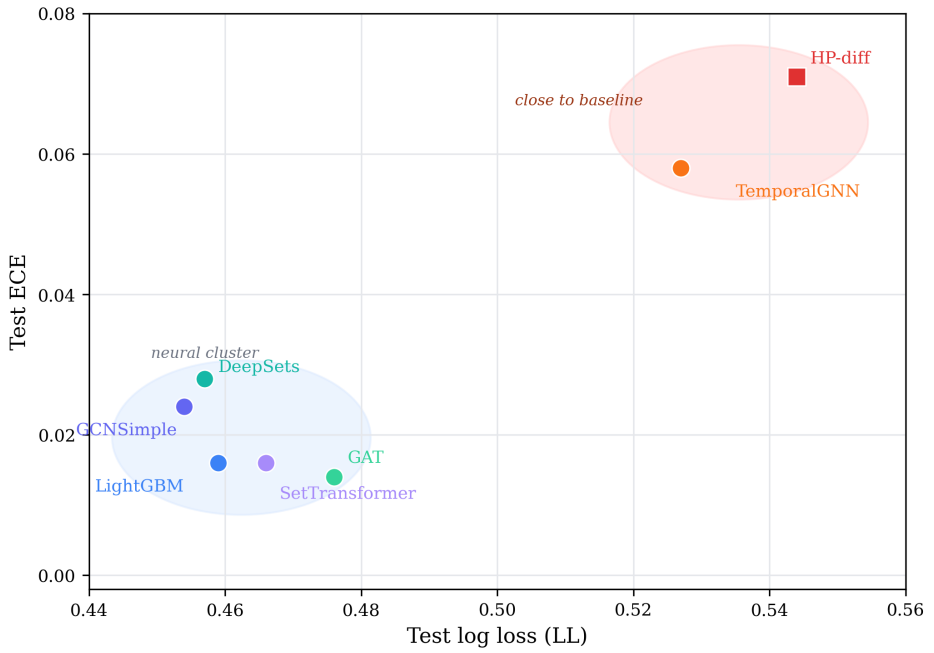


Figure 5.2: Test set log loss and expected calibration error (ECE) for all models on `de_dust2`. TemporalGNN sits markedly closer to the HP-diff baseline than to the neural cluster (LightGBM, DeepSets, GCNSimple, GAT, SetTransformer) on both axes, indicating that the frozen-encoder temporal model recovers little signal beyond a simple health-difference heuristic.

The primary cause is architectural: because the GCNSimple encoder is frozen during temporal training, the GRU receives graph embeddings that were not optimised for sequential prediction. Without end-to-end gradient flow, the temporal model cannot learn to produce embeddings that are informative across time steps, and the GRU effectively re-learns a compressed version of the information already available in a single frame. The frozen encoder is a limitation imposed by training compute constraints, not a principled design choice; end-to-end training would require substantially more GPU memory and time per map. This result should be interpreted as a lower bound on the potential of temporal models for this task, not a definitive verdict.

5.3. Phase Analysis

Table 5.5 breaks down test set log loss by round phase. Late pre-plant states are those occurring at least 85 seconds after round start; post-plant states follow a successful bomb plant. These phases are selected because later states are closer to a decisive outcome: as the round timer expires or the bomb countdown runs, the space of possible outcomes contracts and well-calibrated models should assign more confident, accurate probabilities.

The two phases introduce qualitatively different dynamics. In the late pre-plant phase, both the round timer and the threat of a CT win via time running out create pressure on the T side to commit to a site execute; the game state increasingly reflects the consequences of earlier positioning and utility usage. Post-plant states flip this dynamic: once the bomb is planted, the CT side must reach and defuse within 40 seconds or lose the round regardless of player count or equipment. A non-negligible proportion of professional rounds ends with CTs saving — conceding the round to preserve equipment — rather than attempting a defuse. This makes post-plant prediction qualitatively harder than a simple countdown model would suggest: the model must simultaneously reason about spatial proximity to the bomb, the viability of a defuse attempt, and the CT side's incentive to save.

These phases are selected because later states are closer to a decisive outcome: as the round timer expires or the bomb countdown runs, the space of possible outcomes contracts and well-calibrated models should assign more confident, accurate probabilities. By the late pre-plant and post-plant phases, the round has typically thinned out from the opening 5v5: eliminations accumulate over the first minute, and the surviving player counts — alongside their positioning — become the dominant factors determining the outcome. A 3v1 with the bomb planted is a categorically different prediction problem from a 5v5 mid-round state, and isolating these phases allows a cleaner evaluation of how well each model handles the scenarios where positional and numerical context matters most.

5.3.1. Post-plant advantage of graph models

The most striking result in Table 5.5 is the divergence between graph and set models in the post-plant phase. On `de_dust2`, GCNSimple achieves a post-plant LL of 0.2005, compared to 0.4597 for DeepSets and 0.4713 for SetTransformer. This gap — approximately 0.26 in log loss — does not appear in the full-round results (Table 5.3), where the difference between GCNSimple and DeepSets is only 0.005. The post-plant phase accounts for a minority of

game states in any round (12,883 of 81,544 on the dust2 test set), so its outsized contribution to the full-round gap confirms that relational structure is disproportionately informative after bomb plant.

This result is consistent across maps. GCNSimple achieves the best post-plant LL on six of eight maps; on `de_mirage`, GAT leads (0.277 vs. 0.294), though both graph models substantially outperform set models there too. The set models' post-plant LL is essentially equal to their full-round LL on all maps — they derive no additional signal from the phase transition, because they have no mechanism to reason about spatial configuration.

The explanation is more nuanced than a simple relational vs. non-relational distinction. Graph models encode the spatial configuration of players through pairwise geodesic distance and visibility edges, enabling the model to reason directly about each player's proximity to the bomb site. Set models receive the same per-player (x,y,z) positions as node features, but aggregate them through a permutation-invariant pooling operation that discards the relational structure between players — the model knows where each player is, but not how players are arranged relative to one another or relative to the bomb. LightGBM operates at a coarser level still: the vector representation aggregates all player-level information into team totals, retaining no individual positional signal beyond the binary bomb-site indicator (site A or site B).

LightGBM nonetheless performs competitively in the post-plant phase — matching GCNSimple on `de_nuke` (0.181 vs. 0.182), which suggests that team-level features such as player count, equipment value, and the bomb timer carry substantial post-plant signal even without spatial resolution, and that the graph's relational advantage is most pronounced in geometrically complex situations where individual positioning determines the outcome.

5.3.2. Late pre-plant

In the late pre-plant phase, GCNSimple and LightGBM consistently outperform set models, but the gap between graph and tabular models is smaller than in the post-plant phase. This is expected: late pre-plant states involve strategic positioning ahead of a site execute, which is partially captured by team-level aggregations (equipment value, player count) without requiring pairwise relational reasoning. The spatial configuration of individual players relative to each other matters, but is not as determinative as in post-plant scenarios.

Figure 5.3 illustrates this pattern on `de_mirage`. LightGBM, GCNSimple, and GAT all improve steadily as the round progresses, with log loss declining from around 0.55 in the opening seconds to below 0.35 by t=110s — reflecting the contracting outcome space as the round nears its resolution. DeepSets and SetTransformer show no such trend: both remain near 0.45–0.50 log loss throughout the pre-plant phase, indicating that their representations fail to extract the additional signal available in later-round states. The gap between the two groups widens monotonically with time, suggesting that the information separating a decided round from an open one is precisely the relational and positional context that set-based aggregation discards. This pattern of behaviour held true on all the tested maps.

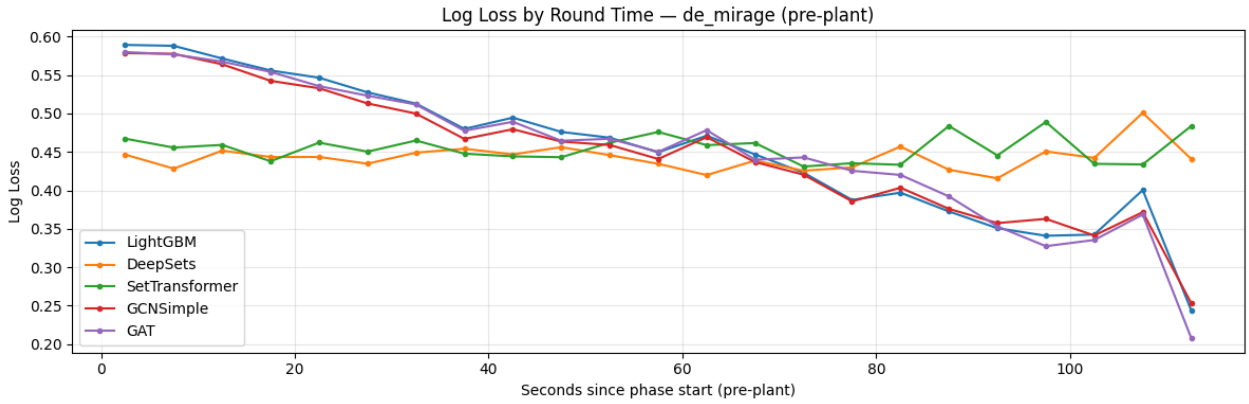


Figure 5.3: Test set log loss on de_mirage grouped into 5s buckets by seconds since phase start

5.4. Qualitative Analysis

This section examines two individual rounds from the test set to ground the aggregate results in concrete tactical situations. Example 1 uses Round 18 of the 3DMAX vs. FaZe Clan match on `de_dust2` to illustrate how graph models exploit spatial and relational information unavailable to tabular and set-based approaches, particularly in the post-plant phase. Example 2 uses Round 21 of the FURIA vs. Natus Vincere match on `de_inferno` to examine model behaviour in a low-probability CT win, and to motivate the use of ECE alongside log loss as a complementary evaluation metric.

5.4.1. Example 1: Positional danger and the limits of tabular models

Figure 5.4 shows the CT win probability trace for Round 18 of the 3DMAX vs. FaZe Clan match on `de_dust2` (StarLadder Major, Stage 3).

Throughout the pre-plant phase, models start close together near 55% CT — reflecting a genuine 5v5 with some early damage already dealt to bodyy (54 hp) and Ex3rcice (43 hp) on the T side. LightGBM and HP-diff climb steadily toward 70–80% CT as the economic advantage of FaZe's full buy accumulates in the feature vector; the graph models are more restrained, hovering in the 55–65% range. At approximately t=55 s, FaZe's in-game leader karrigan is tucked deep inside Pit — a recessed alcove at the bottom of Long A — while 3DMAX's AWPer Maka advances down the same corridor without checking the corner behind him. The resulting configuration is a textbook one-sided visibility window: karrigan has direct line-of-sight to Maka's back; Maka has none to karrigan. This is precisely the kind of relational asymmetry that the graph's directed visibility edges are designed to encode. Both graph models have access to the one-sided visibility edge between karrigan and Maka — encoded as a mean-aggregated node feature in GCNSimple and fed directly into the attention score in GAT. Consistent with this, GAT shows a slight upward nudge in CT probability during this window, while GCNSimple holds approximately flat; whether this reflects the visibility signal specifically or the broader attention weighting of a high-proximity edge cannot be determined from the trace alone. LightGBM, which has no access to individual player

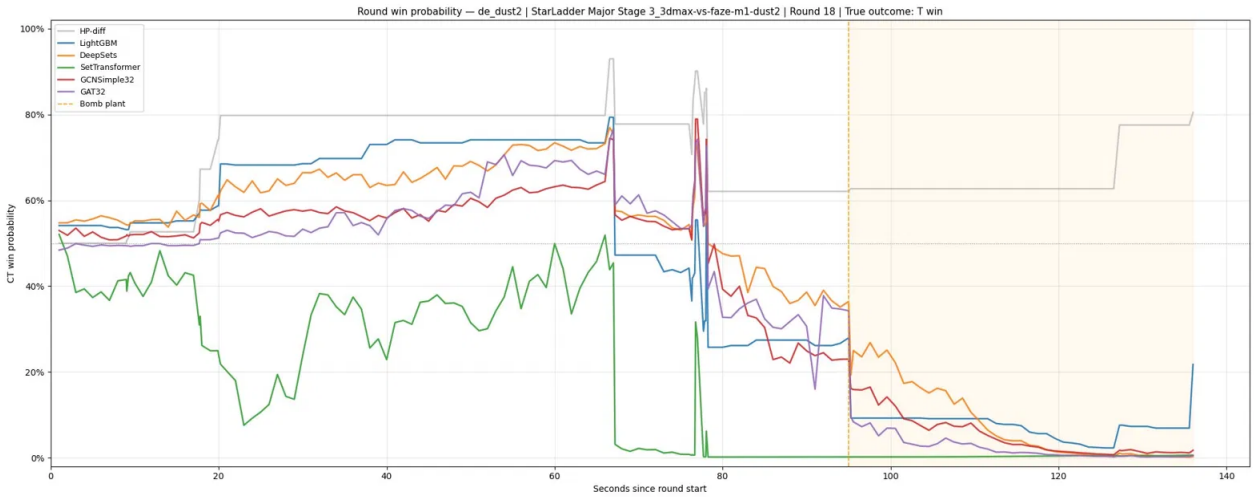


Figure 5.4: CT win probability trace for Round 18 of the 3DMAX vs. FaZe Clan match on `de_dust2` (StarLadder Major, Stage 3).

positions or pairwise visibility, continues unaffected by the positional configuration entirely. The engagement resolves at approximately t=75 s: karrigan fires first, landing a high-damage shot (91 hp, reducing Maka to 9 hp), but fails to secure the kill. Maka instantly turns and eliminates karrigan. All models spike briefly as karrigan's shot registers the health swing, then collapse as the kill is confirmed — the 4v5 deficit is unambiguous regardless of representation. The next few seconds produce a rapid sequence of traded kills in the Middle area of the map: Twistzz (FaZe, CT) eliminates Graviti (3DMAX, T), Lucky (3DMAX, T) eliminates Twistzz, jcobbb (FaZe, CT) eliminates Ex3rcice (3DMAX, T), and bodyy (3DMAX, T) immediately eliminates jcobbb — four kills in quick succession, leaving broky and frozen as the two surviving CTs against Maka (9 hp), bodyy, and Lucky. The Ts plant the bomb at approximately $t = 93$ s (dashed line). The post-plant phase reveals the sharpest model divergence. GCNSimple and GAT drop below 10% CT probability within seconds of the plant and reach near-zero before t=110 s. LightGBM, by contrast, holds at approximately 10% through much of the post-plant phase: it knows the player counts and equipment values but cannot reason about the geometric relationship between the two surviving CTs and the planted bomb. DeepSets behaves similarly, declining slowly from 25% as it has no access to the spatial configuration that makes this 2v3 unlikely to be won.

Facing a 2v3 post-plant with the bomb ticking, the two surviving FaZe CTs — broky and frozen — opt to save their weapons rather than attempt a defuse, retreating to opposite sides of the map. At t≈136 s, broky, positioned around Bombsite B, catches bodyy attempting to exit the taken A bombsite and eliminates him with an AWP shot, momentarily producing a 2v2 state with the bomb still ticking. LightGBM registers a visible probability spike here, responding to the improved player count. GCNSimple does not: the bomb timer has almost run its course, and the positional graph tells the model that the remaining CTs are too far from the bomb site to defuse. The round ends as a T win shortly after.

This example illustrates the core limitation of tabular and set-based models in the post-plant phase: they observe *who* is alive and how healthy they are, but not *where* they are relative to the bomb. Graph models, through their distance and visibility edges, encode exactly

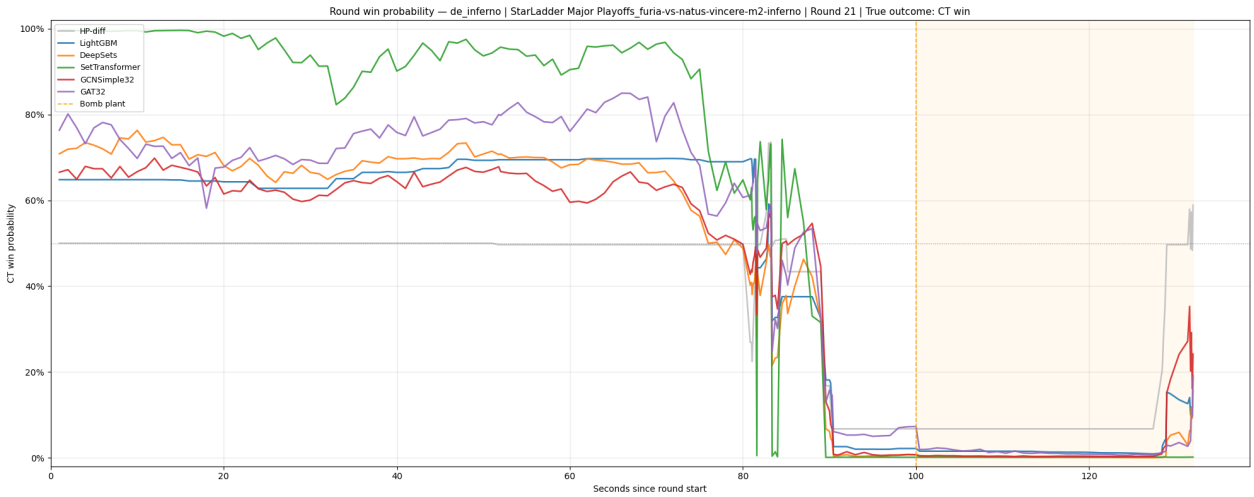


Figure 5.5: CT win probability trace for Round 21 of the FURIA vs. Natus Vincere match on `de_inferno` (StarLadder Major Playoffs).

this relational information, which is why their post-plant log loss advantage over set models (0.20 vs. 0.46 on `de_dust2`, Table 5.5) is as large as it is.

5.4.2. Example 2: A low-probability CT win and the motivation for ECE

Figure 5.5 shows Round 21 of the FURIA vs. Natus Vincere match on `de_inferno` (StarLadder Major Playoffs). The true outcome is a CT win — specifically, a 1v3 post-plant clutch by FalleN. FURIA are the CT side with a full buy (average equipment value 6,170 per player); NAVI are on a force buy round with upgraded pistols (average 2,040 per player).

Pre-plant (0–82 s), the economic asymmetry is immediately reflected in all models. LightGBM, DeepSets, and GCNSimple all track at 65–75% CT probability throughout this phase, consistent with a force-buy scenario where the CT equipment advantage is decisive. SetTransformer, characteristically, diverges erratically — reaching near 100% for much of the first 80 seconds before collapsing without apparent trigger, illustrating its instability rather than any tactical signal.

At $t \approx 82$ s, the round changes. NAVI's w0nderful and FURIA's yuurih exchange kills simultaneously on Bombsite A; b1t then eliminates KSCERATO seconds later. By $t \approx 90$ s, NAVI's makazze has eliminated YEKINDAR and molodoy in rapid succession, reducing FURIA to a single surviving player: FalleN, having to rotate from Bombsite B with an M4A1-S. NAVI plant the bomb at $t \approx 101$ s with three players alive (makazze, iM, Aleksib) against FalleN alone. The equipment advantage that FURIA held at the start of the round has also evaporated: the three surviving NAVI players have picked up rifles from the opponents they eliminated, negating the economic edge that drove the early CT-favourable predictions.

All models register the collapse immediately: GCNSimple, GAT, LightGBM, and DeepSets all drop to the 0–5% CT range at the moment the 1v3 post-plant state is established and remain near zero for the next 28 seconds. This is the probabilistically correct assessment: a 1v3 post-plant clutch on a professional server is an extremely rare outcome, and models

assigning near-zero CT probability are, in expectation, well-calibrated.

What the trace cannot show — and what log loss will not penalise — is that FalleN improbably clutches the round. 2 players decide to exit the taken bombsite through Library, and a last second kill onto Aleksib left FalleN just enough time (0.5s) to secure the bomb defusal. GCNSimple registers a brief spike at each elimination but never exceeds approximately 33% CT probability even as the round narrows to a 1v2 and finally a last-second defuse. LightGBM barely registers the kills at all, remaining essentially at 0%.

This example illustrates why log loss alone is an incomplete evaluation metric. A model that assigns 3% CT probability to a state where a 1v3 clutch is executed will incur a log loss penalty of $-\log(0.03) \approx 3.5$ for that game state — a large single-state penalty that is nonetheless appropriate given the prior probability of the outcome. ECE, however, captures whether the model's 3% predictions are collectively correct: if the CT side wins approximately 3% of the time it is assigned 3% probability, the model is well-calibrated even when it is wrong in any individual case. A model that assigned 50% probability to this state would incur a lower log loss penalty on this specific round but would be systematically miscalibrated across all similar situations. The relevant question is not whether the model predicted this particular clutch, but whether its probability estimates are reliable enough to support downstream use — for example, in computing round-level win probability added (WPA) for player valuation.

5.5. Ablation Studies

This section examines two targeted ablation experiments on `de_dust2` to quantify the contribution of specific modelling choices to overall performance. The first ablation varies the size of the training corpus to assess how much data the graph models require to reach competitive performance. The second ablation varies the edge feature configuration of GCNSimple to isolate the contribution of geodesic distance, visibility, and same-team information.

5.5.1. Ablation study: Training set size

To assess the sensitivity of the graph-based models to the amount of training data available, an additional ablation study was conducted on `de_dust2`, the map with the second largest training corpus in this work (313 matches). The training set was partitioned into nine cumulative subsets by incrementally adding tournament groups in chronological order, beginning with 34 matches from the first two Season 1 tournaments and expanding to the full 310-match training set used in the main experiments. At each step, GCNSimple and GAT were trained from a fresh random initialisation under identical hyperparameters to those used in the main experiments, and evaluated on the same fixed held-out test set of 31 matches. The final data point in Figure 5.6 corresponds to the full training set result reported in Section 5.2.

Figure 5.6 reports test log loss and ECE as a function of training set size. The most striking observation is how rapidly both models approach their performance ceilings. GCNSimple attains a test log loss of 0.491 at just 34 matches and reaches near-plateau performance by 115 matches (0.465), a point at which less than 40% of the full training corpus has been used. The addition of the remaining 195 matches yields a total improvement of only 0.013 in log loss. GAT converges even more abruptly: its loss drops from 0.524 at 34 matches to

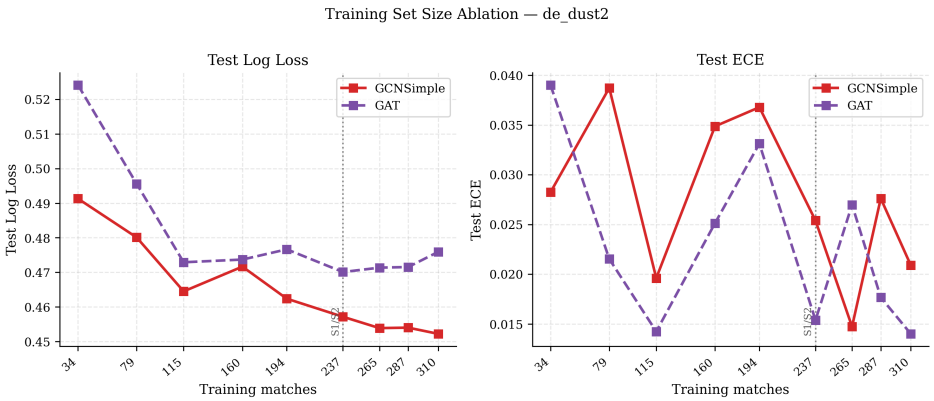


Figure 5.6: Test log loss (left) and ECE (right) for GCNSimple and GAT on `de_dust2` as a function of training set size.

0.473 at 115 matches, after which the curve is essentially flat across all subsequent steps (0.473 → 0.466 from 115 to 310 matches). This early saturation suggests that the attention mechanism in GAT does not exploit additional training data as effectively as the simpler graph convolution, which continues to improve gradually through the Season 1 corpus before stabilising. GCNSimple leads GAT at every data point from 115 matches onward, with the final gap at 310 matches standing at 0.024 log loss (0.452 vs. 0.476).

The ECE curves present a less coherent picture. Both models exhibit high variance across steps with no monotonic improvement trend visible for either architecture. GCNSimple achieves its best calibration at 265 matches (ECE = 0.015) while GAT does so at 115 matches (ECE = 0.014), but neither value is meaningfully distinguished from surrounding steps given the small size of the test set (31 matches, 703 rounds). The oscillatory behaviour is consistent with calibration estimates being inherently high-variance at this evaluation scale, and no reliable conclusion about data efficiency can be drawn from ECE alone in this setting.

Taken together, these results indicate that the graph models trained in this work are not data-hungry in the sense typically assumed of deep neural architectures. A corpus of approximately 115 professional matches appears sufficient to achieve near-peak log loss performance on `de_dust2`. This finding has a direct practical implication for maps with smaller available datasets: the ablation results suggest that competitive graph model performance may be achievable even on maps such as `de_overpass`, which has only 67 training matches — a dataset size at which this ablation shows both models already performing within 0.04 log loss of their respective peaks on `de_dust2`.

5.5.2. Ablation Study: Edge Features

To assess the contribution of each edge feature type, three variants of GCNSimple are trained on `de_dust2`, holding all hyperparameters fixed and varying only the information available on each edge. The *full* model uses all three edge features: inverse geodesic distance as the GCNConv scalar weight, and mean-aggregated visibility and same-team indicators appended to each node's feature vector. The *distance-only* variant zeroes the visibility and same-team contributions before aggregation, retaining only the distance weight. The *no-edges*

variant replaces all edge weights with a uniform value of 1.0 and removes node augmentation entirely, leaving the graph with no relational signal beyond adjacency structure. Results are shown in Figure 5.7.

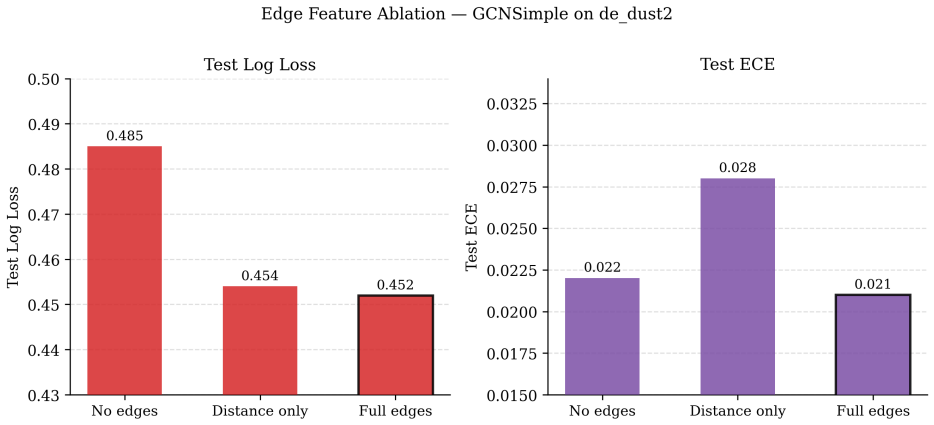


Figure 5.7: Test log loss (left) and ECE (right) for GCNSimple on `de_dust2` as a function of how much information the edges carry. The outlined bar indicates the best-performing configuration.

The largest performance gap lies between the no-edges and distance-only configurations ($\Delta\text{LL} = 0.031$), confirming that geodesic distance is the primary relational signal in the graph. Removing it entirely — reducing message passing to uniform neighbourhood aggregation — degrades performance to 0.485, which is worse than the unregularised DeepSets baseline (0.457) on the same test set and within 0.03 of the HP-diff baseline. This confirms that without meaningful edge weights, the graph structure provides no useful inductive bias over a simpler set-based aggregation.

The additional contribution of visibility and same-team edges is smaller ($\Delta\text{LL} = 0.002$), though consistent in direction. As discussed in Section 5.3, these features are expected to carry disproportionate weight in post-plant states, where line-of-sight to the bomb location is tactically decisive; the full-round aggregate masks this effect. Section 5.5.2 illustrates a round in which the spatial proximity of the two surviving CT players — encoded through distance edges — leads the distance-weighted variants to rate the CT side's chances more favourably than the no-edges model during a critical pre-plant window.

The ECE values across configurations (0.021–0.028) do not follow a consistent pattern and should not be over-interpreted: at this scale, ECE is sensitive to bin boundary effects, and the differences are within the range of run-to-run variance expected from the stochastic training procedure.

Example 3: Edge features in the post-plant graph

Figure 5.8 shows the CT win probability traces for Round 18 of the 3DMAX vs. FaZe Clan match on `de_dust2`, comparing the three GCNSimple edge variants and GAT.

The most notable difference between variants occurs during the 2-vs-3 pre-plant phase (approximately $t = 60$–75 s), where the no-edges model assigns lower CT win probability

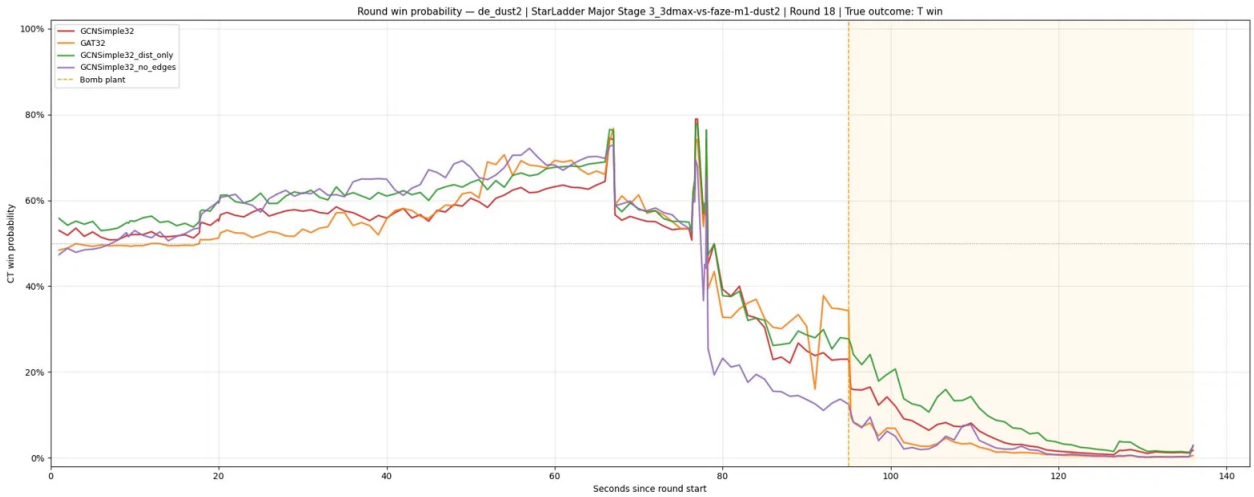


Figure 5.8: CT win probability trace for Round 18 of the 3DMAX vs. FaZe Clan match on `de_dust2` (StarLadder Major, Stage 3) — graph models only.

than the distance-weighted variants. The two surviving CTs were in close proximity to one another during this window — a configuration that, from a tactical standpoint, increases their ability to coordinate a push or trade. The distance-weighted models encode this proximity directly through edge weights and consequently rate the CT side's chances more favourably than a model that treats all spatial configurations as equivalent. All four models converge to near-zero CT probability by the end of the round, consistent with the T-side victory.

Table 5.1: Dataset split summary across all eight maps. Game states are counted at the sampled tick rate after downsampling. Single-season maps (`de_anubis`, `de_overpass`) use a chronological within-season split; all other maps use the StarLadder Major as the held-out test set. The split column reports the percentage of matches allocated to each partition.

Map	Split	Matches	Rounds	Game states	Split %
de_dust2	Train	313	6,724	767,541	77%
	Val	60	1,286	144,808	15%
	Test	31	703	81,544	8%
de_mirage	Train	290	6,357	704,443	76%
	Val	61	1,437	156,790	16%
	Test	30	655	72,335	8%
de_train	Train	140	3,117	345,937	68%
	Val	42	954	105,868	20%
	Test	24	488	53,019	12%
de_nuke	Train	269	5,953	667,841	73%
	Val	75	1,572	176,987	20%
	Test	24	510	57,299	7%
de_ancient	Train	273	5,940	644,171	79%
	Val	53	1,142	123,469	15%
	Test	19	421	47,153	6%
de_inferno	Train	322	7,030	854,837	75%
	Val	79	1,736	209,241	18%
	Test	28	573	70,640	7%
de_anubis*	Train	107	2,293	269,470	69%
	Val	27	576	66,053	17%
	Test	22	451	52,445	14%
de_overpass*	Train	67	1,478	176,551	68%
	Val	18	359	42,231	18%
	Test	14	308	37,006	14%

* Single-season maps using a chronological within-season split.

Table 5.2: CT win rate on the held-out test set for each map, alongside the train-set CT win rate and the Constant baseline log loss. Maps are ordered by test CT win rate.

Map	CT win rate (train)	CT win rate (test)	Constant LL
de_anubis	0.413	0.441	9.007
de_dust2	0.459	0.469	8.557
de_ancient	0.491	0.455	8.781
de_inferno	0.467	0.496	8.128
de_overpass	0.523	0.511	7.875
de_mirage	0.488	0.521	7.727
de_train	0.536	0.522	7.704
de_nuke	0.518	0.534	7.515

Table 5.3: Full-round test set log loss (LL) for all models across all eight maps. Bold indicates the best model per map. Overpass results marked † are unreliable due to the small training set (67 matches).

Model	dust2	mirage	train	nuke	ancient	anubis	overpass	inferno
Constant	8.5568	7.7265	7.7035	7.5146	8.7805	9.0073	7.8752	8.1284
HP-diff	0.5444	0.5336	0.5178	0.5025	0.5415	0.5390	0.5216	0.5490
LightGBM	0.4590	0.4543	0.4562	0.4559	**0.4724**	0.4567	**0.4922**†	0.4659
DeepSets	0.4566	**0.4415**	**0.4463**	0.4644	0.4816	0.4552	0.5284†	0.4651
SetTransformer	0.4662	0.4549	0.4649	0.4900	0.5022	0.4934	0.5939†	0.4892
GCNSimple	**0.4522**	0.4539	0.4494	**0.4512**	0.4739	**0.4521**	0.5055†	**0.4642**
GAT	0.4759	0.4552	0.4494	0.4757	0.4899	0.4631	0.5154†	0.4825

Table 5.4: Full-round test set expected calibration error (ECE) for all models across all eight maps. Lower is better. Bold indicates the best neural model per map. Overpass results marked † are unreliable due to the small training set (67 matches).

Model	dust2	mirage	train	nuke	ancient	anubis	overpass	inferno
Constant	0.5309	0.4794	0.4779	0.4662	0.5448	0.5588	0.4886	0.5043
HP-diff	0.0708	0.0369	0.0385	0.0397	0.0732	0.0755	0.0384	0.0557
LightGBM	0.0158	0.0255	0.0288	0.0250	0.0376	0.0361	**0.0311**†	**0.0195**
DeepSets	0.0276	**0.0120**	0.0221	0.0271	0.0448	0.0254	0.0369†	0.0241
SetTransformer	0.0158	0.0272	0.0306	0.0412	0.0690	0.0411	0.1141†	0.0443
GCNSimple	0.0209	0.0215	0.0139	0.0211	0.0344	**0.0177**	0.0455†	0.0288
GAT	**0.0140**	0.0193	**0.0174**	**0.0210**	**0.0206**	0.0216	0.0419†	0.0236

Table 5.5: Per-phase test set log loss across all maps. Late pre-plant states occur at least 85 s after round start; post-plant states follow a successful bomb plant. Set models (DeepSets, SetTransformer) are included to illustrate their inability to improve in either phase. Bold indicates the best model per map per phase. Overpass omitted from interpretation; see Section 5.2.

Phase	Model	dust2	mirage	train	nuke	ancient	anubis	overpass	inferno
Late pre-plant	LightGBM	0.3444	0.3521	0.3806	**0.3230**	0.3918	0.3819	**0.3857**†	**0.3824**
	DeepSets	0.4419	0.4369	0.4341	0.4514	0.4963	0.4546	0.5434†	0.4536
	SetTransformer	0.4622	0.4648	0.4787	0.4820	0.4839	0.4948	0.6044†	0.4997
	GCNSimple	**0.3339**	0.3526	0.3770	0.3316	**0.3714**	**0.3759**	0.4168†	0.3860
	GAT	0.3912	**0.3438**	**0.3689**	0.3520	0.3903	0.4283	0.4125†	0.4027
Post-plant	LightGBM	0.2303	**0.2590**	0.2285	**0.1810**	0.2738	0.2271	0.2274†	0.2393
	DeepSets	0.4597	0.4414	0.4509	0.4756	0.4877	0.4555	0.5181†	0.4765
	SetTransformer	0.4713	0.4593	0.4477	0.4821	0.4923	0.5115	0.5871†	0.4869
	GCNSimple	**0.2005**	0.2937	**0.1938**	0.1823	**0.2676**	**0.2242**	**0.2132**†	**0.2238**
	GAT	0.2319	0.2766	0.2054	0.1947	0.2678	0.2316	0.2424†	0.2534

6. Discussion

6.1. Log Loss and Calibration as Complementary Objectives

The experiments in Chapter 5 report log loss and ECE as separate metrics, but the relationship between them has a direct implication for how results should be interpreted when choosing a model for downstream use. Log loss penalises confident wrong predictions heavily and rewards confident correct ones; a model that assigns 3% probability to a state where the CT side ultimately wins incurs a penalty of $-\log(0.03) \approx 3.5$, which is appropriate given the prior probability of the outcome. ECE, by contrast, does not care about individual predictions: it asks only whether the model's stated confidence levels are collectively accurate across many game states.

These two objectives can diverge in practice. GAT achieves the best ECE on four of the seven maps with reliable results (`de_dust2`, `de_train`, `de_nuke`, `de_ancient`), yet its full-round log loss is significantly worse than GCNSimple on the same maps according to the Nemenyi post-hoc test ($p = 0.02$). This pattern is consistent with GAT producing smoother, more conservative probability estimates — closer to the true base rate at each confidence level — while GCNSimple makes sharper predictions that are correct more often but incur larger penalties when wrong.

The choice between these profiles depends on the intended application. For computing Win Probability Added (WPA) [18], calibration is the critical property: WPA attributions are derived from differences in win probability between consecutive game states, and a systematically overconfident model will inflate the apparent impact of individual actions, distorting player valuations. The same argument applies to broadcast visualisations, where a probability displayed to viewers must feel honest — a spike to 90% that should be 60% will be visibly wrong to anyone with game knowledge, eroding trust in the metric. For coaching analysis, however, GCNSimple's sharper predictions offer a complementary use case: a model that confidently assigns low probability to an outcome that nevertheless occurs is flagging that round as statistically exceptional — precisely the kind of signal a coaching staff would want to investigate. A CT-side win assigned 8% probability is more analytically interesting than one assigned 35%, even if the former incurs a larger log loss penalty. In this framing, log loss performance is not a deployment criterion but a sensitivity criterion: a model with better log loss is more discriminating about which rounds are genuinely surprising, making it a more useful tool for surfacing tactically significant moments in a match archive.

6.2. Why LightGBM Is Harder to Beat Than Expected

LightGBM operates on the vector representation $\mathbf{v}_t$, which discards player positions entirely and aggregates all spatial information into team-level counts. Intuitively, one would expect that models with access to individual player positions and spatial relationships — the set and graph models — should substantially outperform a positionally-blind ensemble. The Friedman test result ($\chi^2 = 20.92$, $p = 0.0003$) confirms that model differences are not due to chance, but the post-hoc Nemenyi tests show that GCNSimple, DeepSets, and LightGBM form a statistically indistinguishable top group on the full-round prediction task. LightGBM's competitive performance isn't just a statistical artifact, but reflects something real about the structure of the prediction problem.

The most plausible explanation is that economy dominates over positioning for the majority of game states in a round. In the early and mid pre-plant phase — which accounts for the large majority of game states in any round — the equipment differential between teams is the single strongest predictor of the outcome. LightGBM has direct access to this signal through the total equipment value features in $\mathbf{v}_t$, and its gradient-boosted architecture is well-suited to exploiting such tabular, non-linear relationships efficiently. The neural models must reconstruct this signal from individual player equipment values, introducing additional variance without necessarily adding useful information during phases where positioning has not yet become decisive. The full-round log loss, aggregated across all phases, therefore reflects whichever model best handles the economy-dominated majority of states, which LightGBM does with a simplicity that neural models struggle to match at this dataset scale. One of the ways that the performance of the neural models could be improved in future work is by giving them access to fully aggregated values of equipment value, health, and armor, akin to what is currently present in the vector that LightGBM bases its prediction upon.

6.3. The Null Result as a Finding: Where Predictive Signal Lives

The statistical indistinguishability of GCNSimple, DeepSets, and LightGBM on full-round log loss could be read as a negative result for the graph approach. A more accurate interpretation is that it identifies where the actionable predictive signal in CS2 game states actually resides. The full-round metric is dominated by the economy-heavy early and mid-round phases, during which all three models converge to similar predictions. The graph model's advantage concentrates in the post-plant phase, where spatial context is decisive and the relational inductive bias provides a genuine edge: GCNSimple achieves a post-plant log loss of 0.2005 on `de_dust2` compared to 0.4597 for DeepSets and 0.2303 for LightGBM. The aggregate metric obscures this difference because post-plant states are a minority of all game states.

The null result therefore does not mean that relational modelling is uninformative for this task. It means that the task, as evaluated on full-round log loss, is not primarily a spatial reasoning problem for most of its duration. A future evaluation protocol that weights post-plant states more heavily, or that reports phase-conditional metrics as primary results rather

than as supplementary analysis, would better reflect the distinct contribution of graph-based representations.

6.4. Limitations

This work has a number of limitations worth acknowledging. The models are trained exclusively on Tier 1 professional matches, meaning their predictions may not transfer to amateur or semi-professional play, where economy management is less disciplined, positioning is less structured, and individual decision-making is more erratic — in short, the models may struggle to capture the higher degree of randomness characteristic of lower-level gameplay. Player identity is also absent from the feature representation: every alive player on a given side is treated as an interchangeable node, so the model cannot distinguish between a passive, economy-preserving playstyle and an aggressive one. This distinction is particularly relevant in one-vs-N clutch scenarios, where the tactical tendencies of the individual player — not just their health and position — provide important context about the likely outcome. A third simplification is that the models consume no information about the broader match context. The current scoreline, momentum effects such as a N-round losing streak, or the implicit pressure of a match-point situation are all factors that professional players respond to and that may shift round outcomes in ways not captured by the instantaneous game state.

6.4.1. Single-Map Training and Cross-Map Generalisation

All models in this work are trained independently on each map. While this ensures that the learned representations are well-adapted to the geometry and tactical conventions of each individual map, it also means that no information is shared across maps during training. A model trained on `de_dust2` has no knowledge of `de_nuke`, even though the underlying prediction task — estimating round win probability from player states — is identical. This approach is expensive: it requires a separate training run per map, and maps with small training corpora such as `de_overpass` (67 matches) receive no benefit from the larger datasets available for other maps.

A natural extension would be to train a single shared model across all maps, conditioning on map identity either through a learned map embedding or through map-specific normalisation of the spatial features. This would allow the model to transfer tactical knowledge across maps — for example, learning that geodesic proximity to a bomb site is predictive of post-plant outcomes regardless of which map the round takes place on.

6.4.2. Static Graph and Absence of Temporal Reasoning

The graph G_t is constructed independently at each game state tick. No information about the trajectory of the round — player movement or the sequence of eliminations — is propagated between consecutive states. Each game state is therefore evaluated in isolation, without context about how the current configuration was reached.

This is a significant representational limitation. In practice, the direction a player is moving, whether a CT is rotating from one site to another, or whether a T-side execute is

in progress are all relevant to the probability estimate at any given tick. A spatio-temporal extension of the graph model — adding temporal edges between consecutive player nodes across ticks, or processing sequences of per-tick graphs through a recurrent readout — would allow the model to reason over trajectories rather than snapshots. This represents the most direct architectural improvement available to future work.

6.4.3. Overpass as an Unreliable Data Point

The `de_overpass` results are excluded from comparative analysis throughout this work due to the instability of training on only 67 matches. Validation loss was non-monotone during model selection for all neural architectures, and SetTransformer's ECE of 0.114 on this map is more than three times its worst performance on any other map. These results are reported for completeness but should not be interpreted as evidence about model behaviour on `de_overpass` specifically. The data efficiency ablation in Section 5.5.1 suggests that approximately 115 matches are sufficient for graph models to reach near-peak performance on `de_dust2`; future data collection for `de_overpass` should aim to meet this threshold before results on that map are considered reliable.

7. Conclusion

This thesis investigated round win probability prediction in professional Counter-Strike 2 using a systematic comparison of established and novel model architectures. The primary contributions are threefold: a large-scale dataset, a new evaluation methodology, and a controlled multi-model comparison grounded in prior work.

The dataset comprises over 2,000 professional CS2 matches collected across eight maps and two competitive seasons. Rather than pooling all available data into a single split, a tournament-boundary split was proposed in which the held-out test set consists of a single major tournament — the StarLadder Major — ensuring that test matches are drawn from a distinct competitive event with no overlap with training or validation data. This split methodology is more reflective of real deployment conditions than random round-level splits used in some prior work.

Five model classes were implemented and evaluated: LightGBM, DeepSets, Set Transformer, GAT, and GCNSimple — the last of which was developed as part of this work as a regularised, edge-augmented graph convolutional network adapted to the CS2 game state graph. All models were evaluated using log loss and expected calibration error on a consistent test set across every map, enabling direct comparison with the benchmarks established in prior work [18, 20].

A key contribution of the experimental evaluation is the phase-conditional analysis, which separates full-round performance into late pre-plant and post-plant phases. This decomposition reveals patterns that aggregate metrics obscure: all models perform similarly during the economy-dominated pre-plant phase, while meaningful differences emerge post-plant, where spatial configuration around the bomb site becomes the primary driver of outcome. GCNSimple's post-plant log loss is the best of any model on six of seven maps, suggesting that explicit relational modelling of player positions and line-of-sight relationships carries genuine predictive signal in spatially decisive scenarios — even if this advantage does not translate into statistically significant differences on the full-round metric.

Set Transformer and NNConv both failed to generalise at the dataset scales available per map, representing informative constraints on the viable architecture space for this data regime rather than failures of the underlying approaches.

Future work should explore three directions. Training a shared model across all maps — conditioned on map identity — would make use of the full dataset and allow tactical knowledge to transfer across different map geometries. Adding temporal reasoning, through recurrent readouts or temporal edges between consecutive graph states, would allow the model to reason over round trajectories rather than isolated snapshots. Finally, incorporating player identity embeddings — allowing the model to learn that specific players carry individual predictive signal beyond their in-round statistics — could provide a more personalised and tactically nuanced probability estimate.

Bibliography

[1] A. Bisberg and E. Ferrara. Gcn-wp – semi-supervised graph convolutional networks for win prediction in esports. 07 2022.

[2] L. Chen and W. Wang. Analysis of technical features in basketball video based on deep learning algorithm. *Signal Processing: Image Communication*, 83:115786, 04 2020.

[3] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, 2014.

[4] T. Decroos, L. Bransen, J. Van Haaren, and J. Davis. Actions speak louder than goals: Valuing player actions in soccer. 02 2018.

[5] R. Goka, Y. Moroto, K. Maeda, T. Ogawa, and M. Haseyama. Multimodal shot prediction based on spatial-temporal interaction between players in soccer videos. *Applied Sciences*, 14:4847, 06 2024.

[6] J. Gudmundsson and M. Horton. Spatio-temporal analysis of team sports – a survey. 02 2016.

[7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330. PMLR, 2017.

[8] E. H. L. Q. S. D. G. H. W. W. Jia Y, Anida Abdullah N. A narrative review of deep learning applications in sports performance analysis: current practices, challenges, and future directions. *BMC Sports Sci Med Rehabil.*, 2025.

[9] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. Lightgbm: a highly efficient gradient boosting decision tree. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 3149–3157, Red Hook, NY, USA, 2017. Curran Associates Inc.

[10] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[11] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[12] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 3744–3753. PMLR, 2019.

[13] J. Lee, E. Park, and A. P. del Pobil. We know who wins: graph-oriented approaches of passing networks for predictive football match outcomes. *Journal of Big Data*, 12, 06 2025.

[14] J. Ochin, G. Devineau, B. Stanciulescu, and S. Manitsaris. Game state and spatio-temporal action detection in soccer using graph neural networks and 3d convolutional networks. 02 2025.

[15] M. Rahman. A deep learning framework for football match prediction. *SN Applied Sciences*, 2, 02 2020.

[16] Y. Rong, W. Huang, T. Xu, and J. Huang. Dropedge: Towards deep graph convolutional networks on node classification. In *International Conference on Learning Representations*, 2020.

[17] M. Stöckl, T. Seidl, D. Marley, and P. Power. Making offensive play predictable -using a graph convolutional network to understand defensive performance in soccer. 04 2021.

[18] P. Xenopoulos, H. Doraiswamy, and C. Silva. Valuing player actions in counter-strike: Global offensive, 11 2020.

[19] P. Xenopoulos, W. R. Freeman, and C. Silva. Analyzing the differences between professional and amateur esports through win probability. In *Proceedings of the ACM Web Conference 2022*, pages 3418–3427, 2022.

[20] P. Xenopoulos and C. Silva. Esta: An esports trajectory and action dataset, 09 2022.

[21] P. Xenopoulos and C. Silva. Graph neural networks to predict sports outcomes, 07 2022.

[22] S. Yucebas. A deep learning analysis for the effect of individual player performances on match results. *Neural Computing and Applications*, 34, 08 2022.

[23] R. Yurko, F. Matano, L. Richardson, N. Granered, T. Pospisil, K. Pelechrinis, and S. Ventura. Going deep: Models for continuous-timewithin-play valuation of game outcomesin american football with tracking data. 06 2019.

[24] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[25] J. Zhang, D. Han, S. Han, H. Li, W. Lam, and M. Zhang. Chatmatch: Exploring the potential of hybrid vision–language deep learning approach for the intelligent analysis and inference of racket sports. *Computer Speech and Language*, 89, Jan. 2025. Publisher Copyright: © 2024 The Author(s).

List of Figures

5.1 Gap between validation and training log loss (val LL − train LL) at each Set-Transformer training epoch across all eight maps. A rising gap indicates overfitting: the model memorises training trajectories while failing to generalise. `de_overpass` and `de_anubis` show the most extreme divergence, consistent with their small training sets (67 and 107 matches respectively). `de_nuke` is the only map where the gap remains near zero at epoch 2 before diverging . . . 21
5.2 Test set log loss and expected calibration error (ECE) for all models on `de_dust2`. TemporalGNN sits markedly closer to the HP-diff baseline than to the neural cluster (LightGBM, DeepSets, GCNSimple, GAT, SetTransformer) on both axes, indicating that the frozen-encoder temporal model recovers little signal beyond a simple health-difference heuristic. 21
5.3 Test set log loss on de_mirage grouped into 5s buckets by seconds since phase start 24
5.4 CT win probability trace for Round 18 of the 3DMAX vs. FaZe Clan match on `de_dust2` (StarLadder Major, Stage 3). 25
5.5 CT win probability trace for Round 21 of the FURIA vs. Natus Vincere match on `de_inferno` (StarLadder Major Playoffs). 26
5.6 Test log loss (left) and ECE (right) for GCNSimple and GAT on `de_dust2` as a function of training set size. 28
5.7 Test log loss (left) and ECE (right) for GCNSimple on `de_dust2` as a function of how much information the edges carry. The outlined bar indicates the best-performing configuration. 29
5.8 CT win probability trace for Round 18 of the 3DMAX vs. FaZe Clan match on `de_dust2` (StarLadder Major, Stage 3) — graph models only. 30

List of Tables

3.1 Per-map dataset statistics. Maps active for only one season are annotated accordingly. 9

4.1 Player-level feature vector $\mathbf{x}_t^p \in \mathbb{R}^{32}$. All continuous features are normalised prior to training. 13
4.2 Graph model regularisation configuration (`GRAPH_CFG`) applied to GCNSimple and GAT across all maps. 18

5.1 Dataset split summary across all eight maps. Game states are counted at the sampled tick rate after downsampling. Single-season maps (`de_anubis`, `de_overpass`) use a chronological within-season split; all other maps use the StarLadder Major as the held-out test set. The split column reports the percentage of matches allocated to each partition. 31
5.2 CT win rate on the held-out test set for each map, alongside the train-set CT win rate and the Constant baseline log loss. Maps are ordered by test CT win rate. 32
5.3 Full-round test set log loss (LL) for all models across all eight maps. Bold indicates the best model per map. Overpass results marked † are unreliable due to the small training set (67 matches). 32
5.4 Full-round test set expected calibration error (ECE) for all models across all eight maps. Lower is better. Bold indicates the best neural model per map. Overpass results marked † are unreliable due to the small training set (67 matches). 32
5.5 Per-phase test set log loss across all maps. Late pre-plant states occur at least 85 s after round start; post-plant states follow a successful bomb plant. Set models (DeepSets, SetTransformer) are included to illustrate their inability to improve in either phase. Bold indicates the best model per map per phase. Overpass omitted from interpretation; see Section 5.2. 33